

1. คำสั่ง Linux สำหรับจัดการและติดตามโปรเซสหรือ job

ส่วนนี้ทดลองคำสั่งที่ใช้ดูสถานะ ปรับลำดับความสำคัญ และยุติโปรเซสบน Ubuntu WSL เพื่อเปรียบเทียบผลที่เกิดขึ้นจริงกับแนวคิดของระบบปฏิบัติการ

1.1 ความแตกต่างระหว่างคำสั่ง ps และ top

ประเด็น	ps	top
ลักษณะข้อมูล	ภาพนิ่ง ณ เวลาที่เรียกคำสั่ง	ข้อมูลสดที่ปรับปรุงต่อเนื่อง
การใช้งานหลัก	ค้นหา กรอง หรือส่งผลต่อให้คำสั่งอื่น	ติดตาม CPU, memory, load และโปรเซสแบบ interactive
การจบคำสั่ง	แสดงผลแล้วคืน prompt ทันที	กด q เพื่อออกจากหน้าจอ top

```
auty@MSI:~$ ps
  PID TTY          TIME CMD
   290 pts/0        00:00:00 bash
  3916 pts/0        00:00:00 ps
```

ภาพที่ 1 ผลคำสั่ง ps ซึ่งแสดงโปรเซสของ terminal ณ ขณะนั้น

```
auty@MSI:/mnt/c/Users/User/Documents/ChatGPT/CPE_333/CPE333_Lab3_solution$ top
top - 15:35:36 up 6:33, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 26 total, 1 running, 25 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7793.2 total, 7165.9 free, 533.0 used, 253.4 buff/cache
top - 15:35:47 up 6:33, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 26 total, 1 running, 25 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7793.2 total, 7165.9 free, 533.0 used, 253.4 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 7260.2 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   21804 12364 9164  S   0.0   0.2   0:04.24 systemd
    2 root        20   0   3120  1920  1920  S   0.0   0.0   0:00.20 init-systemd(Ub
    6 root        20   0   3120  1792  1792  S   0.0   0.0   0:00.00 init
   39 root       19  -1  50432 15488 14720  S   0.0   0.2   0:03.45 systemd-journal
   88 root        20   0   25280 6400  4864  S   0.0   0.1   0:09.19 systemd-udev
  149 systemd+   20   0   21336 12672 10624  S   0.0   0.2   0:00.62 systemd-resolve
  150 systemd+   20   0   91028 7552  6784  S   0.0   0.1   0:00.77 systemd-timesyn
  165 root        20   0   4236  2432  2304  S   0.0   0.0   0:00.17 cron
  166 message+   20   0   9636  4608  4224  S   0.0   0.1   0:00.86 dbus-daemon
  173 root        20   0   17960 8320  7424  S   0.0   0.1   0:00.43 systemd-logind
  177 root        20   0   3160  1920  1792  S   0.0   0.0   0:00.01 agetty
  184 syslog     20   0  222508 5376  4480  S   0.0   0.1   0:00.45 rsyslogd
  187 root        20   0   3116  1792  1664  S   0.0   0.0   0:00.00 agetty
  198 root        20   0  107012 22272 13056  S   0.0   0.3   0:00.09 unattended-upgr
  288 root        20   0   3136   900   768  S   0.0   0.0   0:00.00 SessionLeader
  289 root        20   0   3136  1164  1024  S   0.0   0.0   0:00.06 Relay(290)
  290 auty       20   0   6072  5248  3584  S   0.0   0.1   0:00.12 bash
  291 root        20   0   6692  4224  3584  S   0.0   0.1   0:00.00 login
  334 auty       20   0  20324 10880 8960  S   0.0   0.1   0:00.41 systemd
  335 auty       20   0  21164  3520  1792  S   0.0   0.0   0:00.00 (sd-pam)
  349 auty       20   0   6072  4736  3328  S   0.0   0.1   0:00.00 bash
 1580 polkitd    20   0  308164 7680  6912  S   0.0   0.1   0:00.86 polkitd
 3625 root        20   0  66936 33656 16004  S   0.0   0.4   0:00.88 python3
 3638 _apt        20   0  23432 14720 10496  S   0.0   0.2   0:00.03 https
 3682 _apt        20   0  13100  6656  6144  S   0.0   0.1   0:00.00 gpgv
 4034 auty       20   0   9412  5504  3328  R   0.0   0.1   0:00.11 top
```

ภาพที่ 2 ผลคำสั่ง top ซึ่งแสดงข้อมูลระบบและรายการโปรเซสแบบต่อเนื่อง

1.2 จุดประสงค์และการใช้งานคำสั่ง nice

คำสั่ง `nice` ใช้กำหนดค่า `niceness` ให้โปรแกรมตั้งแต่เริ่มทำงาน ค่านี้มีช่วงโดยทั่วไปตั้งแต่ -20 ถึง 19 โดยค่าที่มากขึ้นหมายถึงลำดับความสำคัญในการใช้ CPU ต่ำลง ตัวอย่าง `nice -n 10 sleep 1000 &` เริ่มโปรแกรม `sleep` ด้วยค่า `nice` เท่ากับ 10 ส่วนโปรแกรมที่กำลังทำงานอยู่แล้วสามารถใช้ `renice` ปรับค่าได้

```
nice -n 10 sleep 1000 &
jobs -l
ps -o pid,ni,stat,comm -C sleep
```

```
auty@MSI:/mnt/c/Users/User/Documents/ChatGPT/CPE 333/CPE333_Lab3_solution$ nice -n 10 sleep 1000 &
jobs -l
ps -o pid,ni,stat,comm -C sleep
[1] 4038
[1]+  4038  Running                  nice -n 10 sleep 1000 &
    PID NI STAT COMMAND
    4038 10 SN  sleep
```

ภาพที่ 3 การเริ่ม job ด้วย nice และตรวจพบค่า NI เท่ากับ 10

ผลทดลองแสดง job หมายเลข 1 มี PID 4038 และสถานะ Running ขณะที่คำสั่ง `ps` แสดงค่า NI เป็น 10 จึงยืนยันว่าค่า `niceness` ถูกกำหนดให้โปรเซสตั้งแต่เริ่มทำงาน

1.3 การยุติโปรเซสและ job

คำสั่ง `kill` ส่ง signal ไปยังโปรเซส โดย `kill PID` อ้างอิงด้วยหมายเลขโปรเซส และ `kill %1` อ้างอิง job หมายเลข 1 ของ shell ค่าเริ่มต้นคือ `SIGTERM` ซึ่งเปิดโอกาสให้โปรแกรมปิดตัวอย่างเรียบร้อย ส่วน `SIGKILL` หรือ -9 ควรใช้เฉพาะเมื่อ `SIGTERM` ไม่ได้ผล

```
nice -n 10 sleep 1000 &
jobs -l
kill %1
sleep 1
jobs -l
```

```
auty@MSI:/mnt/c/Users/User/Documents/ChatGPT/CPE 333/CPE333_Lab3_solution$ nice -n 10 sleep 1000 &
jobs -l
kill %1
sleep 1
jobs -l
[1] 4045
[1]+  4045  Running                  nice -n 10 sleep 1000 &
[1]+  Terminated                  nice -n 10 sleep 1000
```

ภาพที่ 4 การยุติ job หมายเลข 1 ซึ่งเปลี่ยนจาก Running เป็น Terminated

ก่อนสั่ง `kill` ระบบแสดง PID 4045 อยู่ในสถานะ Running หลังส่ง `SIGTERM` และรอหนึ่งวินาที shell แจ้ง Terminated แสดงว่า job ถูกยุติและไม่คงอยู่ในรายการงานของ shell อีกต่อไป

2. การทดลอง foreground, background และ job control

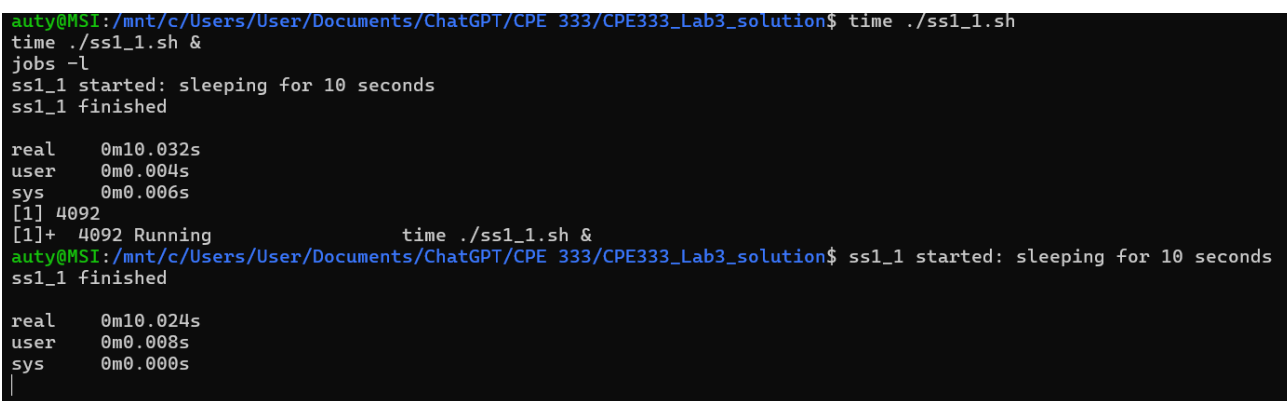
2.1 สถานการณ์ที่ 1: รัน ss1_1.sh แบบปกติและแบบ background

Source code: **ss1_1.sh**

```
#!/usr/bin/env bash

echo "ss1_1 started: sleeping for 10 seconds"
sleep 10
echo "ss1_1 finished"
```

ทดลองครั้งแรกด้วย `time ./ss1_1.sh` ซึ่งรันใน foreground จากนั้นทดลอง `time ./ss1_1.sh &` เพื่อให้ shell สร้าง background job และตรวจสอบด้วย `jobs -l`



```
auty@MSI:/mnt/c/Users/User/Documents/ChatGPT/CPE 333/CPE333_Lab3_solution$ time ./ss1_1.sh
time ./ss1_1.sh &
jobs -l
ss1_1 started: sleeping for 10 seconds
ss1_1 finished

real    0m10.032s
user    0m0.004s
sys     0m0.006s
[1] 4092
[1]+  4092 Running                  time ./ss1_1.sh &
auty@MSI:/mnt/c/Users/User/Documents/ChatGPT/CPE 333/CPE333_Lab3_solution$ ss1_1 started: sleeping for 10 seconds
ss1_1 finished

real    0m10.024s
user    0m0.008s
sys     0m0.000s
```

ภาพที่ 5 เปรียบเทียบการรัน ss1_1.sh แบบ foreground และ background

อภิปรายผล

การรันแบบ foreground ใช้เวลาประมาณ 10.032 วินาทีและ shell ไม่คืน prompt จนกว่า script จะเสร็จ ขณะที่การเติมเครื่องหมาย & ทำให้ shell รายงาน job หมายเลข 1 พร้อม PID 4092 และคืน prompt ทันที แม้ script จะยังทำงานต่อใน background จนครบประมาณ 10.024 วินาที ดังนั้นความแตกต่างสำคัญคือการครอบครอง terminal ไม่ใช่ระยะเวลาของคำสั่ง sleep

2.2 สถานการณ์ที่ 2: Ctrl+Z, jobs, bg และ fg

Source code: `ss1_2.sh`

```
#!/usr/bin/env bash

echo "ss1_2 started: sleeping for 1000 seconds"
sleep 1000
echo "ss1_2 finished"
```

เริ่มด้วย `./ss1_2.sh` แล้วกด Ctrl+Z ขณะ script อยู่ใน foreground จากนั้นใช้ `jobs -l` ตรวจสอบสถานะ ใช้ `bg %1` ให้ job ทำงานต่อด้านหลัง และใช้ `fg %1` นำ job กลับมาอยู่ด้านหน้า

```
auty@MSI:/mnt/c/Users/User/Documents/ChatGPT/CPE 333/CPE333_Lab3_solution$ jobs -l
bg %1
jobs -l
fg %1
[1]+  4052 Stopped                  ./ss1_2.sh
[1]+  ./ss1_2.sh &
[1]+  4052 Running                  ./ss1_2.sh &
./ss1_2.sh
```

ภาพที่ 6 สถานะ Stopped และการทำงานต่อด้วย bg และ fg

คำสั่ง/การกระทำ	ผลที่เกิดขึ้น
Ctrl+Z	shell ส่ง SIGTSTP ไปยัง foreground process group ทำให้ job เป็น Stopped
jobs -l	แสดงหมายเลข job, PID และสถานะปัจจุบัน
bg %1	ส่ง SIGCONT ให้ job ทำงานต่อใน background และคืน prompt ให้ผู้ใช้
fg %1	นำ job กลับสู่ foreground ทำให้ terminal รอจนงานเสร็จหรือถูกหยุดอีกครั้ง

อภิปรายผล

ภาพทดลองแสดง job หมายเลข 1 เปลี่ยนจาก Stopped หลัง Ctrl+Z เป็น Running หลัง bg เมื่อใช้ fg ชื่อคำสั่ง `./ss1_2.sh` ปรากฏด้านหน้าและ shell ไม่คืน prompt แสดงว่า job กลับมาครอบครอง terminal จึงสรุปได้ว่า bg และ fg ต่างทำให้งานดำเนินต่อเหมือนกัน แต่ว่างกันที่ตำแหน่งการทำงานและการควบคุม terminal

5. การจำลอง STCF Scheduler ด้วยภาษา C

STCF (Shortest Time-to-Completion First) เป็นการจัดตารางแบบ preemptive ซึ่งเลือกโปรเซสที่มีเวลาทำงานเหลือน้อยที่สุด เมื่อมีโปรเซสใหม่มาถึง scheduler จะประเมิน ready queue ใหม่ และสามารถแย่ง CPU จากโปรเซสเดิมได้

5.1 ข้อกำหนดและข้อมูลที่ใช้

เงื่อนไขจากโจทย์	การนำไปใช้ในโปรแกรม
โปรเซสมาถึงเวลา T	นำเข้า ready queue ก่อน scheduler เลือกงานสำหรับ time slot T
มาถึงพร้อมกัน	ใช้ process_id เป็นเกณฑ์จัดลำดับร่วมกับกฎ STCF
remaining time เท่ากัน	เลือก process_id ที่มีค่าน้อยกว่า
context switch เร็วมาก	ไม่นับเวลา scheduler และ context switch เป็น CPU usage
ไม่มีโปรเซสพร้อมทำงาน	แสดงช่วงเวลาเป็น IDLE

5.2 ภาพรวมการทำงานของฟังก์ชัน simulate_stfc()

ลำดับ	การทำงาน
1	เริ่มเวลา time = 0 และนับจำนวนโปรเซสที่เสร็จแล้ว
2	ในแต่ละ time slot ค้นหาโปรเซสที่ arrival <= time และยังไม่ finished
3	เลือก remaining ที่น้อยที่สุด หากเท่ากันเลือก PID ที่น้อยกว่า
4	ให้โปรเซสที่เลือกใช้ CPU หนึ่งหน่วยเวลาแล้วลด remaining ลงหนึ่ง
5	เมื่อ remaining เป็นศูนย์ให้ทำเครื่องหมาย finished และเพิ่ม completed
6	รวม time slot ที่ CPU owner เดิมทำงานต่อเนื่องเป็นช่วง [start-end] ก่อนพิมพ์ผล

การเลือกโปรเซสใหม่เกิดขึ้นทุกหนึ่งหน่วยเวลา จึงรองรับ preemption โดยธรรมชาติ ตัวแปร selected เก็บ index ของงานที่ดีที่สุด ส่วน current_pid และ segment_start ใช้รวมผลลัพธ์ที่ต่อเนื่องเพื่อให้อ่านง่าย

5.2.1 ฟังก์ชัน simulate_stfc() - ส่วนที่ 1

```
1 void simulate_stfc(Process *procs, int n) {
2     const int IDLE = -1;
3     int time = 0;
4     int completed = 0;
5     int current_pid = IDLE;
6     int segment_start = 0;
7     int segment_started = 0;
8
9     if (n <= 0) {
10         printf("No processes to schedule.\n");
11         return;
12     }
13
14     /* A zero/negative burst needs no CPU time. */
15     for (int i = 0; i < n; i++) {
16         if (procs[i].remaining <= 0) {
17             procs[i].remaining = 0;
18             procs[i].finished = 1;
19             completed++;
20         }
21     }
22
23     while (completed < n) {
24         int selected = -1;
25
26         /*
27          * Choose the arrived process with the shortest remaining
28          * time.
29          * A smaller process ID wins when remaining times are equal.
30          */
31         for (int i = 0; i < n; i++) {
32             if (procs[i].finished || procs[i].arrival > time) {
33                 continue;
34             }
35
36             if (selected == -1 ||
37                 procs[i].remaining < procs[selected].remaining ||
38                 (procs[i].remaining == procs[selected].remaining
39                  &&
40                   procs[i].pid < procs[selected].pid)) {
41                 selected = i;
42             }
43         }
44
45         if (selected != -1) {
46             time = procs[selected].arrival;
47             segment_start = time;
48             segment_started = 0;
49             procs[selected].remaining--;
50             completed++;
51         }
52     }
53 }
```

ภาพที่ 7 ฟังก์ชัน simulate_stfc() ส่วนประกาศตัวแปรและการเลือกโพรเซส

5.2.2 ฟังก์ชัน simulate_stfc() - ส่วนที่ 2

```
40     }
41 }
42
43     int next_pid = (selected == -1) ? IDLE : procs[selected].
pid;
44
45     /* Print a completed segment only when the CPU owner changes. */
46     if (!segment_started) {
47         current_pid = next_pid;
48         segment_start = time;
49         segment_started = 1;
50     } else if (next_pid != current_pid) {
51         if (current_pid == IDLE) {
52             printf("[%d-%d]: IDLE\n", segment_start, time);
53         } else {
54             printf("[%d-%d]: P%d\n", segment_start, time,
current_pid);
55         }
56         current_pid = next_pid;
57         segment_start = time;
58     }
59
60     /* One loop iteration represents one CPU time slot. */
61     if (selected != -1) {
62         procs[selected].remaining--;
63         if (procs[selected].remaining == 0) {
64             procs[selected].finished = 1;
65             completed++;
66         }
67     }
68
69     time++;
70 }
71
72     if (segment_started) {
73         if (current_pid == IDLE) {
74             printf("[%d-%d]: IDLE\n", segment_start, time);
75         } else {
76             printf("[%d-%d]: P%d\n", segment_start, time,
current_pid);
77         }
78     }
79 }
```

ภาพที่ 8 ฟังก์ชัน simulate_stfc() ส่วนบันทึกช่วงเวลาและอัปเดต remaining time

5.3 ผลการทดสอบทั้ง 3 cases

```
auty@MSI:/mnt/c/Users/User/Documents/ChatGPT/CPE 333/CPE333_Lab3_solution$ ./PS3 case1.csv
[0-2]: P1
[2-4]: P2
[4-5]: P3
[5-7]: P2
[7-12]: P1
auty@MSI:/mnt/c/Users/User/Documents/ChatGPT/CPE 333/CPE333_Lab3_solution$ ./PS3 case2.csv
[0-2]: P1
[2-5]: IDLE
[5-6]: P2
[6-7]: P4
[7-9]: P2
[9-12]: P3
auty@MSI:/mnt/c/Users/User/Documents/ChatGPT/CPE 333/CPE333_Lab3_solution$ ./PS3 case3.csv
[0-3]: P2
[3-5]: P1
[5-7]: P2
[7-8]: P3
```

ภาพที่ 9 ผลลัพธ์ของ case1.csv, case2.csv และ case3.csv

5.3.1 Case 1: การ preempt ต่อเนื่อง

P1 เริ่มทำงานช่วง [0-2] และเหลือเวลา 5 หน่วย เมื่อ P2 มาถึงเวลา 2 ด้วย burst 4 จึงแย่ง CPU จาก P1 ต่อมา P3 มาถึงเวลา 4 ด้วย burst 1 จึงแย่ง CPU จาก P2 หลัง P3 เสร็จ scheduler กลับไปเลือก P2 ซึ่งเหลือ 2 หน่วย ก่อนปิดท้ายด้วย P1 ผลลัพธ์ [0-2] P1, [2-4] P2, [4-5] P3, [5-7] P2 และ [7-12] P1 จึงสอดคล้องกับ STCF

5.3.2 Case 2: ช่วง IDLE และการตัดสินใจเมื่อมาถึงพร้อมกัน

P1 เสร็จที่เวลา 2 แต่โปรเซสยังไปมาถึงเวลา 5 จึงเกิด IDLE ช่วง [2-5] เมื่อ P2 และ P3 มาถึงพร้อมกันและมี burst เท่ากัน ระบบเลือก P2 เพราะ PID ต่ำกว่า จากนั้น P4 มาถึงเวลา 6 ด้วย remaining 1 จึง preempt P2 หลัง P4 เสร็จจึงกลับไป P2 และสุดท้าย P3

5.3.3 Case 3: remaining time เท่ากัน

P2 ทำงานตั้งแต่ [0-3] และเหลือเวลา 2 หน่วย เมื่อ P1 มาถึงเวลา 3 ด้วย burst 2 ค่า remaining จึงเท่ากันและเลือก P1 เพราะ PID ต่ำกว่า หลัง P1 เสร็จ P2 ทำงานต่อ ช่วงเวลา 6 P3 มาถึงด้วย remaining 1 เท่ากับ P2 แต่ P2 มี PID ต่ำกว่าและกำลังทำงานอยู่จึงทำต่อจนเสร็จ ก่อนให้ P3 ทำงานช่วง [7-8] ผลนี้ยืนยันว่ากฎ tie-break ด้วย process_id ถูกใช้อย่างถูกต้อง

5.4 สรุปผล

โปรแกรมสามารถอ่านข้อมูลจาก CSV เลือกโปรเซสตาม STCF รองรับการ preempt แสดงช่วง IDLE และตัดสินใจเลือกด้วย process_id ได้ครบทั้งสามกรณีทดสอบ ผลที่พิมพ์ออกมาสอดคล้องกับลำดับเหตุการณ์ของแต่ละ time slot และข้อสมมติที่กำหนดในใบงาน